جزوه

Frontend

فرانت‌اند: React، Angular، Blazor و Vue کنار هم

1. Components: input, output and state · کامپوننت: ورودی، رویداد، و وضعیت
2. Calling the API with tokens · فراخوانی API با توکن: یک refresh مشترک برای همه‌ی ۴۰۱ها
3. Route guards and roles · محافظ مسیر: صفحه‌ی ادمین برای Viewer باز نمی‌شود
4. Forms and client-side validation · فرم و اعتبارسنجی سمت کلاینت، با همان قواعد سرور

درس 1 · frontend

Components: input, output and state

کامپوننت: ورودی، رویداد، و وضعیت

components props events state

بعد از این درس می‌توانی

- یک کامپوننت کارت با ورودی (پروژه) و خروجی (رویداد refresh) در چهار فریم‌ورک بسازی.
- وضعیت (فهرست، در حال بارگذاری، خطا) را در کامپوننت والد نگه داری و تغییرش بدهی.
- بفهمی هر فریم‌ورک کی دوباره render می‌کند: Blazor بعد از event، Angular با signal، React با setState، Vue با ref.

سناریو از پروژه‌ی واقعی

داشبورد Control Panel برای هر پروژه یک کارت دارد: سالم یا خراب، یک خلاصه، و دکمه‌ای برای خواندن دوباره‌ی وضعیت. در این درس همان کارت و فهرست را در Blazor (زبانی که از #C می‌آید)، Angular (فرانت Control Panel)، React (فرانت ClinicReport و ReBiomed) و Vue (همین سایت) می‌سازیم. الگو یکی است: داده از بالا به پایین با props/input، رویداد از پایین به بالا با callback/output.

سؤال چرا کارت خودش وضعیت را دوباره نمی‌خواند؟

پاسخ کارت فقط نمایش می‌دهد و می‌گوید «دکمه زده شد». والد تصمیم می‌گیرد چه کند و فهرست را عوض می‌کند. این‌طوری کارت در هر صفحه‌ای قابل استفاده است و فقط یک جا داده را عوض می‌کند.

سؤال در React چرا آرایه را map می‌کنیم و push نمی‌کنیم؟

پاسخ React تغییر را با مقایسه‌ی مرجع تشخیص می‌دهد. اگر همان آرایه را تغییر بدهی، مرجع عوض نشده و ممکن است render نشود. Vue و Angular signal تغییر درجا را هم می‌بینند، ولی الگوی آرایه‌ی جدید در همه درست کار می‌کند.

مفهوم‌ها

ورودی

Blazor: [Parameter] روی property. Angular 17+: input.required<T>() که یک signal است. React: props تابع. Vue: defineProps. در هر چهار، کامپوننت نباید ورودی را مستقیم تغییر بدهد.

خروجی

Blazor: EventCallback<T> و Angular: output<T>() و React: یک تابع در props (onRefresh). Vue: defineEmits و emit('refresh', key).

وضعیت و render

Blazor بعد از هر event handler و بعد از await در lifecycle خودش render می‌کند. Angular با signal فقط جاهایی که signal خوانده شده را به‌روز می‌کند. React با هر setState کل تابع کامپوننت را دوباره اجرا می‌کند. Vue تغییر ref/reactive را ردیابی می‌کند و فقط وابسته‌ها را به‌روز می‌کند.

بارگذاری اولیه

Blazor: OnInitializedAsync. Angular: ngOnInit. React: useEffect با آرایه‌ی وابستگی خالی (و پرچم cancelled برای وقتی کامپوننت قبل از پاسخ unmount شد). Vue: onMounted.

کلید در فهرست

Blazor @key، Angular track، React key، Vue :key. فریم‌ورک با کلید می‌فهمد کدام کارت کدام است و فقط همان را به‌روز می‌کند.

مقایسه

ورودی	
Blazor (C#)	`[Parameter] public ProjectStatus Project { get; set; }`
Angular	`project = input.required<ProjectStatus>()`
React	`function StatusCard({ project }: Props)`
Vue	`defineProps<{ project: ProjectStatus }>()`

در Angular ورودی یک signal است و با project() خوانده می‌شود.

رویداد به والد	
Blazor (C#)	`EventCallback<string> OnRefresh + InvokeAsync`
Angular	`refresh = output<string>() + emit`
React	`onRefresh: (key: string) => void`
Vue	`defineEmits<{ refresh: [key: string] }>()`

React رویداد ندارد؛ فقط تابعی که والد پاس داده.

وضعیت	
Blazor (C#)	`private List<ProjectStatus>? projects;`
Angular	`projects = signal<ProjectStatus[]>([])`
React	`const [projects, setProjects] = useState(...)`
Vue	`const projects = ref<ProjectStatus[] | null>(null)`

Blazor فیلد معمولی است؛ بقیه یک ظرف واکنشی دارند.

مقدار محاسبه‌شده	
Blazor (C#)	`private string Label => ...`
Angular	`label = computed(() => ...)`
React	`const label = ...` (در هر render)
Vue	`const label = computed(() => ...)`

React در هر render دوباره حساب می‌کند؛ برای هزینه‌بر useMemo.

حلقه با کلید

Blazor (C#)	`@foreach + @key`
Angular	`@for (p of projects(); track p.key)`
React	`projects.map(p => <StatusCard key={p.key} />)`
Vue	`v-for="p in projects" :key="p.key"`

Angular از نسخه‌ی 17 track را اجباری کرده.

بارگذاری اولیه

Blazor (C#)	`OnInitializedAsync()`
Angular	`ngOnInit()`
React	`useEffect(() => {...}, [])`
Vue	`onMounted(async () => ...)`

useEffect در حالت StrictMode در development دو بار اجرا می‌شود.

کد

کارت وضعیت

Blazor (C#) کد اجراشده و تست‌شده در /examples

examples/frontend/blazor/Lessons.Blazor/StatusCard.razor

```
<article class="card @(Project.Healthy ? "" : "down")">
    <h3>@Project.DisplayName</h3>
    <p>@Label</p>
    <button @onclick="() => OnRefresh.InvokeAsync(Project.Key)" disabled="@Busy">Refresh</button>
</article>

@code {
    [Parameter, EditorRequired] public ProjectStatus Project { get; set; } = default!; // in: Project="p"
    [Parameter] public bool Busy { get; set; }
    [Parameter] public EventCallback<string> OnRefresh { get; set; } // out: OnRefresh="Reload"

    private string Label => Project.Healthy ? Project.Summary ?? "Healthy" : Project.Error ?? "Down";
}
```

Angular کد اجراشده و تست‌شده در /examples examples/frontend/web/angular/status-card.component.ts

```ts
@Component({
  selector: 'app-status-card',
  standalone: true,
  template: `
    <article class="card" [class.down]="!project().healthy">
      <h3>{{ project().displayName }}</h3>
      <p>{{ label() }}</p>
      <button (click)="refresh.emit(project().key)" [disabled]="busy()">Refresh</button>
    </article>
  `,
})
export class StatusCardComponent {
  project = input.required<ProjectStatus>() // in: [project]="p"
  busy = input(false)
  refresh = output<string>() // out: (refresh)="reload($event)"
  // Recomputed only when project() changes.
  label = computed(() => {
    const p = this.project()
    return p.healthy ? (p.summary ?? 'Healthy') : (p.error ?? 'Down')
  })
}
```

React کد اجراشده و تست‌شده در /examples examples/frontend/web/react/StatusCard.tsx

```tsx
interface Props {
  project: ProjectStatus // in: <StatusCard project={p} />
  busy?: boolean
  onRefresh(key: string): void // out: a callback prop, React's version of an event
}

export function StatusCard({ project, busy = false, onRefresh }: Props) {
  const label = project.healthy ? (project.summary ?? 'Healthy') : (project.error ?? 'Down')
  return (
    <article className={project.healthy ? 'card' : 'card down'}>
      <h3>{project.displayName}</h3>
      <p>{label}</p>
      <button onClick={() => onRefresh(project.key)} disabled={busy}>
        Refresh
      </button>
    </article>
  )
}
```

Vue کد اجراشده و تست‌شده در /examples examples/frontend/web/vue/StatusCard.vue

```
<script setup lang="ts">
import { computed } from 'vue'
import type { ProjectStatus } from '../shared/model'

const props = withDefaults(defineProps<{ project: ProjectStatus; busy?: boolean }>(), { busy: false }) // in: :project="p"
const emit = defineEmits<{ refresh: [key: string] }>() // out: @refresh="reload"

const label = computed(() =>
  props.project.healthy ? (props.project.summary ?? 'Healthy') : (props.project.error ?? 'Down'),
)
</script>

<template>
  <article class="card" :class="{ down: !project.healthy }">
    <h3>{{ project.displayName }}</h3>
    <p>{{ label }}</p>
    <button :disabled="busy" @click="emit('refresh', project.key)">Refresh</button>
  </article>
</template>
```

فهرست با وضعیت

Blazor (C#) کد اجراشده و تست‌شده در /examples

examples/frontend/blazor/Lessons.Blazor/StatusList.razor

```
@inject ProjectsClient Api

@if (error is not null)
{
    <p role="alert">@error</p>
}
else if (projects is null)
{
    <p>Loading…</p>
}
else
{
    @foreach (var p in projects)
    {
        <StatusCard @key="p.Key" Project="p" Busy="busyKey == p.Key" OnRefresh="Reload" />
    }
}

@code {
    // Plain fields are the state; Blazor re-renders after an event handler or StateHasChanged().
    private List<ProjectStatus>? projects;
    private string? busyKey;
    private string? error;

    protected override async Task OnInitializedAsync()
    {
        try { projects = await Api.ListAsync(); }
        catch (Exception ex) { error = ex.Message; }
    }

    private async Task Reload(string key)
    {
        busyKey = key;
        try
        {
            var fresh = await Api.StatusAsync(key);
            projects = projects?.Select(p => p.Key == key ? fresh : p).ToList();
        }
        catch (Exception ex) { error = ex.Message; }
        finally { busyKey = null; }
    }
}
```

Angular — کد اجراشده و تست‌شده در /examples — examples/frontend/web/angular/status-card.component.ts

```
@Component({
  selector: 'app-status-list',
  standalone: true,
  imports: [StatusCardComponent],
  template: `
    @if (error()) { <p role="alert">{{ error() }}</p> }
    @for (p of projects(); track p.key) {
      <app-status-card [project]="p" [busy]="busyKey() === p.key" (refresh)="reload($event)" />
    } @empty { <p>Loading…</p> }
  `,
})
export class StatusListComponent implements OnInit {
  private api = inject(ProjectsService)
  // State lives in signals; the template updates when they change.
  projects = signal<ProjectStatus[]>([])
  busyKey = signal<string | null>(null)
  error = signal<string | null>(null)

  ngOnInit() {
    this.api.list().subscribe({ next: (ps) => this.projects.set(ps), error: (e) => this.error.set(e.message) })
  }

  reload(key: string) {
    this.busyKey.set(key)
    this.api
      .status(key)
      .pipe(finalize(() => this.busyKey.set(null)))
      .subscribe({
        next: (fresh) => this.projects.update((ps) => ps.map((p) => (p.key === key ? fresh : p))),
        error: (e) => this.error.set(e.message),
      })
  }
}
```

React کد اجراشده و تست‌شده در /examples examples/frontend/web/react/StatusCard.tsx

```tsx
export function StatusList() {
  // State: React re-renders the component when a setter is called with a new value.
  const [projects, setProjects] = useState<ProjectStatus[] | null>(null)
  const [busyKey, setBusyKey] = useState<string | null>(null)
  const [error, setError] = useState<string | null>(null)

  // Runs after the first render; the empty array means "once", not on every render.
  useEffect(() => {
    let cancelled = false // the component may unmount before the request finishes
    apiFetch<ProjectStatus[]>('/projects')
      .then((ps) => !cancelled && setProjects(ps))
      .catch((e: Error) => !cancelled && setError(e.message))
    return () => {
      cancelled = true
    }
  }, [])

  async function reload(key: string) {
    setBusyKey(key)
    try {
      const fresh = await apiFetch<ProjectStatus>(`/projects/${encodeURIComponent(key)}`)
      // Never mutate state in place: build a new array.
      setProjects((ps) => ps?.map((p) => (p.key === key ? fresh : p)) ?? null)
    } catch (e) {
      setError((e as Error).message)
    } finally {
      setBusyKey(null)
    }
  }

  if (error) return <p role="alert">{error}</p>
  if (!projects) return <p>Loading…</p>
  return (
    <>
      {projects.map((p) => (
        <StatusCard key={p.key} project={p} busy={busyKey === p.key} onRefresh={reload} />
      ))}
    </>
  )
}
```

Vue کد اجراشده و تست‌شده در /examples `examples/frontend/web/vue/StatusList.vue`

```vue
<script setup lang="ts">
import { onMounted, ref } from 'vue'
import type { ProjectStatus } from '../shared/model'
import { apiFetch } from './session'
import StatusCard from './StatusCard.vue'

// ref(): reactive state; the template re-renders when .value changes.
const projects = ref<ProjectStatus[] | null>(null)
const busyKey = ref<string | null>(null)
const error = ref<string | null>(null)

onMounted(async () => {
  try {
    projects.value = await apiFetch<ProjectStatus[]>('/projects')
  } catch (e) {
    error.value = (e as Error).message
  }
})

async function reload(key: string) {
  busyKey.value = key
  try {
    const fresh = await apiFetch<ProjectStatus>(`/projects/${encodeURIComponent(key)}`)
    projects.value = projects.value?.map((p) => (p.key === key ? fresh : p)) ?? null
  } catch (e) {
    error.value = (e as Error).message
  } finally {
    busyKey.value = null
  }
}
</script>

<template>
  <p v-if="error" role="alert">{{ error }}</p>
  <p v-else-if="!projects">Loading…</p>
  <StatusCard v-for="p in projects" v-else :key="p.key" :project="p" :busy="busyKey === p.key"
@refresh="reload" />
</template>
```

اشتباه‌های رایج

React **تغییر درجای state (projects.push، p.healthy = true)**

چرا: مرجع عوض نمی‌شود و React ممکن است render نکند.

درست: همیشه آرایه/شیء جدید: map، spread.

React **useEffect بدون آرایه‌ی وابستگی برای fetch**

چرا: بعد از هر render اجرا می‌شود؛ fetch → setState → render → fetch، حلقه‌ی بی‌پایان.

درست: [] برای یک بار، یا وابستگی‌های دقیق.

Angular **فراموش کردن پرانتز در template: project.healthy به‌جای project().healthy**

چرا: signal یک تابع است؛ بدون () خود تابع را می‌خوانی.

درست: project().healthy.

Vue **destructure کردن props یا reactive**

چرا: قبل از Vue 3.5 واکنش‌پذیری از دست می‌رفت.

درست: props.project یا toRefs؛ از 3.5 destructure در defineProps واکنشی است.

Blazor (C#) **تغییر state از یک thread یا timer بیرونی بدون StateHasChanged**

چرا: Blazor فقط بعد از event handler و lifecycle خودش render می‌کند.

درست: InvokeAsync(StateHasChanged).

این کد کجای پروژه‌هاست

`ClinicReport/web/src/pages/Login.tsx:13`

React: useState در فرانت ClinicReport

`LanguageLessonDesigner/src/components/HeroBlock.vue:4`

Vue: defineProps در همین نوع سایت

`ControlPanel/frontend/src/app/components/assistant-panel/assistant-panel.component.ts:1`

Angular: کامپوننت standalone در Control Panel

تمرین

▼ 1. یک ورودی دوم به کارت اضافه کن: compact (bool) که خلاصه را پنهان کند. در هر فریم‌ورک چند خط است؟

پاسخ: Blazor: یک [Parameter] و یک @if. Angular: compact = input(false) و @if. React: یک prop و {!compact && ...}. Vue: یک prop و v-if. در هر چهار حدود سه خط.

▼ 2. در نسخه‌ی React پرچم cancelled را بردار، و کامپوننت را قبل از پاسخ fetch از صفحه حذف کن. چه می‌شود؟

پاسخ: setProjects روی کامپوننتی که دیگر نیست صدا زده می‌شود. React 18+ هشداری نمی‌دهد ولی کار بیهوده انجام می‌شود، و اگر پاسخ دو درخواست به ترتیب اشتباه برسد داده‌ی قدیمی جای جدید را می‌گیرد.

▼ 3. تست bUnit کارت Blazor را بخوان. معادلش در React با Testing Library چه شکلی است؟

پاسخ: `render(<StatusCard project={p} onRefresh={fn} />); expect(screen.getByText('Adapter unreachable.')); await userEvent.click(screen.getByRole('button')); expect(fn).toHaveBeenCalledWith('lab')`

جمله‌ی مصاحبه

Data flows down through inputs, events flow up through callbacks, and the parent owns the state; only the syntax differs between Blazor, Angular, React and Vue.

درس 2 · frontend

Calling the API with tokens

فراخوانی API با توکن: یک refresh مشترک برای همه‌ی ۴۰۱ها

http | interceptors | token refresh

بعد از این درس می‌توانی

- توکن را در یک جا به همه‌ی درخواست‌ها بچسبانی، نه در هر صفحه.
- وقتی access token منقضی شد (401)، یک بار refresh کنی و درخواست را تکرار کنی، بی‌آنکه کاربر چیزی ببیند.
- وقتی چند درخواست هم‌زمان 401 می‌گیرند فقط یک refresh بفرستی.

سناریو از پروژه‌ی واقعی

در Control Panel، authInterceptor هر درخواست HttpClient را می‌گیرد، توکن را اضافه می‌کند، و در 401 با auth.refreshAccessToken() تلاش می‌کند. refreshAccessToken یک refreshInFlight مشترک دارد، چون سرور refresh token را می‌چرخاند: دو refresh هم‌زمان با همان توکن، دومی را با توکنِ دیگر باطل‌شده می‌فرستد. همین منطق را در Blazor با DelegatingHandler، و برای React و Vue با یک fetch wrapper مشترک ساخته‌ایم. هر دو تست دارند: دو 401 هم‌زمان، دقیقاً یک refresh.

سؤال چرا refresh را در هر صفحه یا hook انجام ندهیم؟

پاسخ چون صفحه‌ها همدیگر را نمی‌شناسند. دو صفحه‌ی هم‌زمان دو refresh می‌فرستند و یکی شکست می‌خورد. یک لایه در مسیر همه‌ی درخواست‌ها (interceptor، handler، wrapper) تنها جایی است که می‌تواند refresh را مشترک کند.

سؤال اگر خود درخواست refresh 401 بگیرد؟

پاسخ یعنی session تمام شده (logout، رمز عوض شده، ۷ روز گذشته). توکن‌ها پاک می‌شوند و کاربر به صفحه‌ی ورود می‌رود. درخواست refresh هرگز از همان مسیر retry رد نمی‌شود، وگرنه حلقه می‌سازد.

مفهوم‌ها

یک لایه برای همه

Angular: HttpInterceptorFn که با provideHttpClient(withInterceptors([...])) ثبت می‌شود. Blazor: DelegatingHandler در pipeline یک HttpClient. React و Vue: فریم‌ورک چیزی ندارد؛ یک wrapper دور fetch (یا interceptor در axios).

refresh مشترک

اولین 401 یک promise/observable/task می‌سازد و در یک فیلد نگه می‌دارد. 401های بعدی همان را await می‌کنند. وقتی تمام شد، فیلد پاک می‌شود. TS. finalize + (1)shareReplay :RxJS: ?? = و C#: lock. finally و یک Task.

تکرار درخواست

Angular و fetch درخواست را دوباره می‌سازند. در .NET یک HttpRequestMessage را نمی‌شود دو بار فرستاد؛ باید یک کپی ساخت.

کجا توکن نگه داریم

این مثال‌ها، مثل localStorage، Control Panel استفاده می‌کنند. یک باگ XSS می‌تواند آن را بخواند. قدم بعدی، cookie از نوع httpOnly برای refresh token است که JavaScript به آن دسترسی ندارد (README خود Control Panel همین را نوشته).

مقایسه

لایه‌ی مشترک	
Blazor (C#)	`class AuthHandler : DelegatingHandler`
Angular	`const authInterceptor: HttpInterceptorFn`
React	`fetch دور createApi(...)`
Vue	`fetch دور createApi(...)`

React و Vue این را ندارند؛ همان کد TypeScript برای هر دو.

ثبت	
Blazor (C#)	`AddHttpClient<ProjectsClient>().AddHttpMessageHandler<AuthHandler>()`
Angular	`provideHttpClient(withInterceptors([authInterceptor]))`
React	`import { apiFetch } from './session'`
Vue	`import { apiFetch } from './session'`

در Blazor و Angular ثبت یک بار است و همه‌ی HttpClientها از آن رد می‌شوند.

refresh مشترک

Blazor (C#)	`lock + Task<string>? refreshing`
Angular	`refreshInFlight + shareReplay(1)`
React	`refreshing ??= (async () => ...)()`
Vue	`refreshing ??= (async () => ...)()`

JavaScript تک‌نخی است و lock لازم ندارد؛ C# چندنخی است.

استفاده در صفحه

Blazor (C#)	`@inject ProjectsClient Api`
Angular	`inject(ProjectsService)`
React	`const { projects } = useProjects()`
Vue	`const { projects } = useProjects()`

hook و composable تقریباً یکی‌اند؛ composable بدون قواعد ترتیب hookها.

کد

توکن و refresh در یک لایه

Blazor (C#) کد اجراشده و تست‌شده در /examples

examples/frontend/blazor/Lessons.Blazor/AuthHandler.cs

```
// Blazor's version of an HttpInterceptor: a DelegatingHandler in the HttpClient pipeline.
// Attach the access token; on 401 refresh once and retry; if the refresh fails, sign out.
public class AuthHandler(ITokenStore tokens, Uri refreshUrl, Action onSignedOut) : DelegatingHandler
{
    // One refresh shared by every request that hit 401 at the same time.
    private Task<string>? refreshing;
    private readonly object gate = new();

    protected override async Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, CancellationToken
ct)
    {
        Attach(request, tokens.Access);
        var response = await base.SendAsync(request, ct);
        if (response.StatusCode != HttpStatusCode.Unauthorized || tokens.Refresh is null) return response;

        string token;
        try
        {
            token = await SharedRefresh(ct);
        }
        catch (Exception)
        {
            tokens.Clear();
            onSignedOut();
            return response;
        }
        response.Dispose();
        // A request can be sent only once: send a copy (the bodies in this app are buffered JSON).
        using var retry = await Clone(request);
        Attach(retry, token);
        return await base.SendAsync(retry, ct);
    }

    private Task<string> SharedRefresh(CancellationToken ct)
    {
        lock (gate)
        {
            refreshing ??= RefreshAsync(ct).ContinueWith(t => { lock (gate) refreshing = null; return t; },
TaskScheduler.Default).Unwrap();
            return refreshing;
        }
    }

    private async Task<string> RefreshAsync(CancellationToken ct)
    {
        // Straight to the inner handler: the refresh call must not come back through this handler.
        using var req = new HttpRequestMessage(HttpMethod.Post, refreshUrl) { Content = JsonContent.Create(new
{ refreshToken = tokens.Refresh }) };
        using var res = await base.SendAsync(req, ct);
        res.EnsureSuccessStatusCode();
        var pair = (await res.Content.ReadFromJsonAsync<TokenPair>(ct))!;
        tokens.Save(pair.AccessToken, pair.RefreshToken);
        return pair.AccessToken;
    }

    private static void Attach(HttpRequestMessage request, string? token)
```

```
    {
        if (token is not null) request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token);
    }

    private static async Task<HttpRequestMessage> Clone(HttpRequestMessage request)
    {
        var copy = new HttpRequestMessage(request.Method, request.RequestUri);
        if (request.Content is not null)
        {
            copy.Content = new ByteArrayContent(await request.Content.ReadAsByteArrayAsync());
            foreach (var h in request.Content.Headers) copy.Content.Headers.TryAddWithoutValidation(h.Key,
h.Value);
        }
        return copy;
    }

    private record TokenPair(string AccessToken, string RefreshToken);
}
```

examples/frontend/web/angular/auth.ts کد اجراشده و تست‌شده در /examples **Angular**

```
const isAuthEndpoint = (req: HttpRequest<unknown>) => req.url.startsWith(`${API_BASE}/auth/`)
const withToken = (req: HttpRequest<unknown>, token: string) => req.clone({ setHeaders: { Authorization:
`Bearer ${token}` } })

// Registered once with provideHttpClient(withInterceptors([authInterceptor])); every HttpClient call passes
through.
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService)
  const router = inject(Router)
  // login/refresh must never trigger another refresh (that would loop).
  if (isAuthEndpoint(req) || !auth.token) return next(req)

  return next(withToken(req, auth.token)).pipe(
    catchError((err: unknown) => {
      if (!(err instanceof HttpErrorResponse) || err.status !== 401 || !auth.hasRefreshToken) return
throwError(() => err)
      return auth.refreshAccessToken().pipe(
        switchMap((token) => next(withToken(req, token))),
        catchError((refreshErr) => {
          auth.clearSession()
          router.navigate(['/login'])
          return throwError(() => refreshErr)
        }),
      )
    }),
  )
}
```

React کد اجراشده و تست‌شده در /examples examples/frontend/web/react/useProjects.ts

```
// A custom hook: components call useProjects() and get data, state and actions.
// The token handling (attach, refresh once on 401, sign out) is inside apiFetch.
export function useProjects() {
  const [projects, setProjects] = useState<ProjectStatus[]>([])
  const [error, setError] = useState<string | null>(null)

  const load = useCallback(async () => {
    try {
      setProjects(await apiFetch<ProjectStatus[]>('/projects'))
    } catch (e) {
      setError((e as Error).message)
    }
  }, [])

  useEffect(() => {
    void load()
  }, [load])

  const create = useCallback(
    async (p: CreateProject) => {
      await apiFetch('/projects', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body:
JSON.stringify(p) })
      await load()
    },
    [load],
  )

  return { projects, error, create, reload: load }
}
```

Vue کد اجراشده و تست‌شده در /examples examples/frontend/web/vue/useProjects.ts

```
// A composable: Vue's version of a custom hook. It runs once per component that calls it.
// The token handling (attach, refresh once on 401, sign out) is inside apiFetch.
export function useProjects() {
  const projects = ref<ProjectStatus[]>([])
  const error = ref<string | null>(null)

  async function load() {
    try {
      projects.value = await apiFetch<ProjectStatus[]>('/projects')
    } catch (e) {
      error.value = (e as Error).message
    }
  }

  async function create(p: CreateProject) {
    await apiFetch('/projects', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body:
JSON.stringify(p) })
    await load()
  }

  onMounted(load)
  return { projects, error, create, reload: load }
}
```

fetch wrapper مشترک React و Vue

همه‌ی زبان‌ها کد اجراشده و تست‌شده در /examples examples/frontend/web/shared/api.ts

```
// A fetch wrapper for React and Vue (Angular does the same in an HttpInterceptor):
// attach the access token; on 401 refresh once and retry; if the refresh fails, sign out.
export function createApi(baseUrl: string, tokens: TokenStore, onSignedOut: () => void, fetchFn = fetch) {
  // One refresh shared by every request that hit 401 at the same time: the server rotates
  // refresh tokens, so two parallel refreshes with the same token would race each other.
  let refreshing: Promise<string> | null = null

  function refreshAccessToken(): Promise<string> {
    refreshing ??= (async () => {
      const res = await fetchFn(`${baseUrl}/auth/refresh`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ refreshToken: tokens.refresh() }),
      })
      if (!res.ok) throw new Error('Session expired.')
      const pair = await res.json()
      tokens.save(pair)
      return pair.accessToken as string
    })().finally(() => (refreshing = null))
    return refreshing
  }

  const withToken = (init: RequestInit, token: string | null): RequestInit => ({
    ...init,
    headers: { ...init.headers, ...(token ? { Authorization: `Bearer ${token}` } : {}) },
  })

  return async function apiFetch<T>(path: string, init: RequestInit = {}): Promise<T> {
    let res = await fetchFn(`${baseUrl}${path}`, withToken(init, tokens.access()))
    if (res.status === 401 && tokens.refresh()) {
      try {
        res = await fetchFn(`${baseUrl}${path}`, withToken(init, await refreshAccessToken()))
      } catch (err) {
        tokens.clear()
        onSignedOut()
        throw err
      }
    }
    if (!res.ok) throw new Error((await res.json().catch(() => null))?.error ?? `HTTP ${res.status}`)
    return res.json() as Promise<T>
  }
}
```

اشتباه‌های رایج

refresh جدا برای هر درخواست 401 همه

چرا: سرور refresh token را می‌چرخاند؛ refresh دوم با توکن قدیمی شکست می‌خورد و کاربر بی‌دلیل خارج می‌شود.

درست: یک refresh در حال اجرا، مشترک بین همه.

همه **رد شدن درخواست refresh از همان interceptor**

چرا: refresh با 401 دوباره refresh را صدا می‌زند: حلقه.

درست: مسیرهای /auth/ را از منطق retry معاف کن (isAuthEndpoint).

Blazor (C#) **فرستادن دوباره‌ی همان HttpRequestMessage**

چرا: InvalidOperationException: درخواست قبلاً فرستاده شده.

درست: یک کپی بساز (Clone).

Angular **subscribe کردن مستقیم به http.post برای refresh بدون shareReplay**

چرا: هر subscriber یک درخواست جدید می‌فرستد؛ observable سرد است.

درست: shareReplay(1).

این کد کجای پروژه‌هاست

`ControlPanel/frontend/src/app/auth/auth.interceptor.ts:18`

authInterceptor: توکن، 401، retry، refresh

`ControlPanel/frontend/src/app/auth/auth.service.ts:43`

refreshInFlight مشترک با shareReplay

`ClinicReport/web/src/api.ts:121`

React: در 401 رویداد signed-out

تمرین

▼ 1. در تست api.test.ts، Promise.all دو درخواست را هم‌زمان می‌فرستد. اگر refreshing ??= را با refreshing = عوض کنی چند refresh می‌بینی؟

پاسخ: دو. هر 401 refresh جدید می‌سازد و تست (refreshes === 1) شکست می‌خورد.

▼ 2. در Blazor، lock را از SharedRefresh بردار. آیا تست همیشه شکست می‌خورد؟

پاسخ: نه همیشه؛ race condition است و به زمان‌بندی threadها بستگی دارد. همین است که باگ هم‌زمانی را خطرناک می‌کند: تست گاهی سبز است.

▼ 3. برای React به‌جای fetch wrapper از axios استفاده کن. معادل interceptor آن چیست؟

پاسخ: axios.interceptors.request.use برای اضافه کردن توکن، و axios.interceptors.response.use با شاخه‌ی خطا برای 401، با همان refresh مشترک.

جمله‌ی مصاحبه

Tokens are attached in one place for every request, and a 401 triggers a single shared refresh before the request is retried.

درس 3 · frontend

Route guards and roles

محافظ مسیر: صفحه‌ی ادمین برای Viewer باز نمی‌شود

routing guards roles

بعد از این درس می‌توانی

- یک صفحه را فقط برای نقش Admin باز کنی و بقیه را به داشبورد برگردانی.
- روش هر فریم‌ورک را بشناسی: [Authorize] در Blazor، canActivate در Angular، wrapper در React، beforeEach در Vue.
- بدانی محافظ فرانت فقط تجربه‌ی کاربری است و امنیت در سرور است.

سناریو از پروژه‌ی واقعی

صفحه‌ی manage-ports در Control Panel را با canActivate: [authGuard, adminGuard] محافظت می‌کند. در ClinicReport همین را با یک شرط در element انجام می‌دهد: can(me, 'users.manage') ? <Users /> : <Navigate to="/" />. LanguageLessonDesigner هم یک router.beforeEach دارد. در هر سه، سرور همان endpointها را جداگانه با نقش چک می‌کند (درس JWT در جزوه‌ی بک‌اند).

سؤال اگر سرور چک می‌کند، guard فرانت چه فایده‌ای دارد؟

پاسخ کاربر Viewer صفحه‌ای را نمی‌بیند که همه‌ی دکمه‌هایش 403 بدهند. guard تجربه است، نه امنیت: هر کسی می‌تواند JavaScript را عوض کند یا API را مستقیم صدا بزند.

سؤال نقش کاربر را فرانت از کجا می‌داند؟

پاسخ از پاسخ login (یا از payload توکن). این مقدار فقط برای نمایش است؛ سرور نقش را از توکن امضاشده می‌خواند، نه از چیزی که کلاینت می‌گوید.

مفهوم‌ها

Blazor: attribute روی صفحه

@attribute [Authorize(Roles = "Admin")] روی صفحه، و AuthorizeRouteView در Router که آن را می‌خواند. AuthenticationStateProvider نقش کاربر را فراهم می‌کند.

Angular: CanActivateFn

یک تابع که true یا یک UrlTree برمی‌گرداند. UrlTree یعنی «به‌جایش برو اینجا». با inject() سرویس‌ها را می‌گیرد.

React Router: کامپوننت wrapper

React Router guard ندارد. یک کامپوننت (RequireRole) یا children را render می‌کند یا <Navigate replace /> . در data routerها می‌شود loader هم استفاده کرد.

Vue Router: guard سراسری با meta

هر route در meta نقش لازم را می‌گوید و یک router.beforeEach همه را چک می‌کند. برگرداندن یک مسیر یعنی redirect. با meta.role، declare module نوع‌دار می‌شود.

مقایسه

علامت زدن صفحه	
Blazor (C#)	`@attribute [Authorize(Roles = "Admin")]`
Angular	`canActivate: [adminGuard]`
React	`<RequireRole role="Admin">...</RequireRole>`
Vue	`meta: { role: 'Admin' }`

Blazor و Vue اعلانی‌اند؛ Angular و React کد را مستقیم در تعریف route می‌گذارند.

جایی که چک می‌شود	
Blazor (C#)	`AuthorizeRouteView`
Angular	`CanActivateFn` خود
React	`wrapper` خود کامپوننت
Vue	`router.beforeEach`

در Vue یک جا برای همه‌ی مسیرها؛ در بقیه برای هر مسیر.

redirect	
Blazor (C#)	`<NotAuthorized>` یا `NavigationManager`
Angular	`return router.createUrlTree(['/dashboard'])`
React	`<Navigate to="/dashboard" replace />`
Vue	`return '/dashboard'`

replace جلوی برگشت با دکمه‌ی Back به صفحه‌ی ممنوع را می‌گیرد.

منبع نقش	
Blazor (C#)	`AuthenticationStateProvider`
Angular	`AuthService.role`
React	`SessionContext`
Vue	`reactive session`

همه باید از پاسخ سرور پر شوند، نه از ورودی کاربر.

کد

Blazor (C#) کد اجراشده و تست‌شده در /examples — `examples/frontend/blazor/Lessons.Blazor/Routes.razor` — فقط **Admin**

```
@using Microsoft.AspNetCore.Components.Authorization
@using Microsoft.AspNetCore.Components.Routing

@* AuthorizeRouteView reads [Authorize] on each page; a page the user may not see never renders. *@
<CascadingAuthenticationState>
    <Router AppAssembly="typeof(Routes).Assembly">
        <Found Context="route">
            <AuthorizeRouteView RouteData="route">
                <NotAuthorized>
                    <p>Only an admin can open this page.</p>
                </NotAuthorized>
            </AuthorizeRouteView>
        </Found>
        <NotFound>
            <p>Not found.</p>
        </NotFound>
    </Router>
</CascadingAuthenticationState>
```

Angular کد اجراشده و تست‌شده در /examples examples/frontend/web/angular/auth.ts

```
export const adminGuard: CanActivateFn = () => {
  const auth = inject(AuthService)
  // Returning a UrlTree redirects; the page never renders for a Viewer.
  return auth.role === 'Admin' ? true : inject(Router).createUrlTree(['/dashboard'])
}

export const routes: Routes = [
  { path: 'dashboard', loadComponent: () => import('./status-card.component').then((m) =>
m.StatusListComponent) },
  {
    path: 'projects/new',
    canActivate: [adminGuard],
    loadComponent: () => import('./project-form.component').then((m) => m.ProjectFormComponent),
  },
  { path: '**', redirectTo: 'dashboard' },
]
```

React کد اجراشده و تست‌شده در /examples examples/frontend/web/react/routes.tsx

```
// React Router has no guard hook: a wrapper component decides what to render.
function RequireRole({ role, children }: { role: Role; children: ReactNode }) {
  const session = useSession()
  return session.role === role ? children : <Navigate to="/dashboard" replace />
}

export function AppRoutes() {
  return (
    <Routes>
      <Route path="/dashboard" element={<StatusList />} />
      <Route
        path="/projects/new"
        element={
          <RequireRole role="Admin">
            <ProjectForm />
          </RequireRole>
        }
      />
      <Route path="*" element={<Navigate to="/dashboard" replace />} />
    </Routes>
  )
}
```

Vue کد اجراشده و تست‌شده در /examples

examples/frontend/web/vue/router.ts

```
declare module 'vue-router' {
  interface RouteMeta {
    role?: Role
  }
}

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/dashboard', component: () => import('./StatusList.vue') },
    { path: '/projects/new', component: () => import('./ProjectForm.vue'), meta: { role: 'Admin' } },
    { path: '/:pathMatch(.*)*', redirect: '/dashboard' },
  ],
})

// One global guard reads each route's meta; returning a path redirects.
router.beforeEach((to) => {
  if (to.meta.role && session.role !== to.meta.role) return '/dashboard'
})
```

صفحه با [Authorize]

Blazor (C#) کد اجراشده و تست‌شده در /examples

examples/frontend/blazor/Lessons.Blazor/NewProjectPage.razor

```
@page "/projects/new"
@using Microsoft.AspNetCore.Authorization
@attribute [Authorize(Roles = "Admin")]

<h2>New project</h2>
<ProjectForm />
```

Angular از کد واقعی پروژه (کوتاه‌شده)

```
export const adminGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn && auth.currentUser?.role === 'Admin') return true;

  router.navigate(['/dashboard']);
  return false;
};

// app.routes.ts
{ path: 'manage-ports', component: ManagePortsPageComponent, canActivate: [authGuard, adminGuard] }
```

React از کد واقعی پروژه (کوتاه‌شده)

```
<Route path="/users" element={can(me, 'users.manage') ? <Users /> : <Navigate to="/" />} />
<Route path="/audit" element={can(me, 'audit.view') ? <Audit /> : <Navigate to="/" />} />
```

Vue از کد واقعی پروژه (کوتاه‌شده)

```
// The private ctx.<domain> copy opens on Context; only the first navigation, so the Lessons tab still works
there.
router.beforeEach((to, from) => {
  if (from === START_LOCATION && to.path === '/' && location.hostname.startsWith('ctx.')) return '/context'
})
```

اشتباه‌های رایج

همه **فقط guard فرانت، بدون چک در سرور**

چرا: هر کسی API را مستقیم صدا می‌زند یا JavaScript را تغییر می‌دهد.

درست: نقش در سرور روی هر endpoint (درس JWT).

Angular **router.navigate داخل guard و return false**

چرا: کار می‌کند ولی دو navigation شروع می‌شود؛ UrlTree تمیزتر است و Angular آن را redirect می‌داند.

درست: return router.createUrlTree([...]).

React **<Navigate> بدون replace**

چرا: صفحه‌ی ممنوع در history می‌ماند و Back دوباره redirect می‌کند.

درست: <Navigate to=... replace /> .

Blazor (C#) **[Authorize] روی صفحه بدون AuthorizeRouteView در Router**

چرا: attribute خوانده نمی‌شود و صفحه برای همه باز است.

درست: AuthorizeRouteView و CascadingAuthenticationState.

این کد کجای پروژه‌هاست

`ControlPanel/frontend/src/app/auth/admin.guard.ts:5`

adminGuard در Angular

`ControlPanel/frontend/src/app/app.routes.ts:14`

canActivate: [authGuard, adminGuard]

`ClinicReport/web/src/App.tsx:78`

React: <Navigate /> : <Page /> ? can(me, ...) </

`LanguageLessonDesigner/src/router/index.ts:41`

Vue: router.beforeEach

تمرین

▼ 1. در نسخه‌ی Vue یک نقش دوم (Viewer) برای /reports تعریف کن، طوری که Admin هم بتواند ببیند. meta و guard چه شکلی می‌شوند؟

پاسخ: { meta: { roles: ['Admin', 'Viewer'] } و در beforeEach: if (to.meta.roles && !to.meta.roles.includes(session.role)) return '/dashboard'.

▼ 2. adminGuard در Control Panel از router.navigate و return false استفاده می‌کند. آن را به UrlTree تبدیل کن.

پاسخ: return auth.isLoggedIn && auth.currentUser?.role === 'Admin' ? true : inject(Router).createUrlTree(['/dashboard']).

▼ 3. با DevTools نقش را در session به Admin تغییر بده و صفحه را باز کن. بعد Save بزن. چه می‌بینی؟

پاسخ: صفحه باز می‌شود (guard فقط سمت کلاینت است) ولی POST /projects با 403 رد می‌شود، چون سرور نقش را از توکن می‌خواند.

جمله‌ی مصاحبه

Route guards keep users away from pages they can't use, but the server checks the role again on every call; the guard is UX, not security.

درس 4 · frontend

Forms and client-side validation

فرم و اعتبارسنجی سمت کلاینت، با همان قواعد سرور

forms | validation | server errors

بعد از این درس می‌توانی

- فرم ساخت پروژه را با همان قواعد DTO بک‌اند در چهار فریم‌ورک بسازی.
- پیام خطا را بعد از اولین تلاش نشان بدهی، نه از لحظه‌ی باز شدن صفحه.
- خطای سرور (مثلاً 409 تکراری) را هم نشان بدهی، چون سرور همیشه دوباره چک می‌کند.

سناریو از پروژه‌ی واقعی

قواعد پروژه در بک‌اند: key با حروف کوچک، عدد و خط تیره تا ۴۰ کاراکتر؛ نام لازم تا ۱۰۰؛ آدرس http(s). فرم فرانت همین قواعد را تکرار می‌کند تا کاربر قبل از ارسال بداند. Blazor می‌تواند همان attributeهای DataAnnotations را روی مدل بگذارد؛ Angular قواعد را روی کنترل‌ها می‌گذارد؛ React و Vue از یک تابع validateProject مشترک استفاده می‌کنند که تست شده است.

سؤال قواعد دو جا نوشته شده‌اند. اگر یکی عوض شود؟

پاسخ سرور مرجع است و همیشه چک می‌کند؛ اگر کلاینت عقب بماند، کاربر 400 می‌بیند و پیام سرور نمایش داده می‌شود. در Blazor می‌شود همان کلاس DTO را بین سرور و کلاینت مشترک کرد، که مزیت #C در هر دو طرف است.

سؤال چرا خطاها را از اول نشان ندهیم؟

پاسخ فرم خالی پر از خطای قرمز است، قبل از اینکه کاربر کاری کرده باشد. بعد از اولین submit (یا touch در Angular) خطاها معنی دارند.

مفهوم‌ها

Blazor: EditForm

EditForm با DataAnnotationsValidator، Model که attributeها را اجرا می‌کند، InputText با @bind-Value، و ValidationMessage برای هر فیلد. OnValidSubmit فقط وقتی همه‌ی قواعد درست باشند صدا زده می‌شود.

Angular: Reactive Forms

FormBuilder.nonNullable.group با Validators روی هر کنترل. markAllAsTouched، control.touched، form.invalid و getRawValue() مقدار typed را می‌دهد.

React: کنترل‌شده

مقدار هر input در state است و onChange آن را عوض می‌کند. خطاها در هر render از state محاسبه می‌شوند (نه در state جدا). برای فرم‌های بزرگ کتابخانه‌هایی مثل react-hook-form با zod.

Vue: v-model

v-model روی یک reactive دوطرفه وصل می‌شود؛ .trim فاصله‌ها را می‌برد. خطاها computed هستند و با هر تغییر فیلد دوباره حساب می‌شوند.

خطای سرور

اعتبارسنجی کلاینت نمی‌داند key تکراری است؛ فقط سرور می‌داند. پاسخ 409 یا 400 را بخوان و پیامش را نشان بده.

مقایسه

تعریف قواعد	
Blazor (C#)	روی مدل [Required, RegularExpression(...)]
Angular	Validators.required, Validators.pattern(...)
React	validateProject(form) (تابع مشترک)
Vue	validateProject(form) (تابع مشترک)

Blazor همان attributeهای سرور C# را استفاده می‌کند.

اتصال input

Blazor (C#)	`<InputText @bind-Value="model.Key" />`
Angular	`<input formControlName="key" />`
React	`<input value={form.key} onChange={set('key')} />`
Vue	`<input v-model.trim="form.key" />`

React تنها جایی است که اتصال دوطرفه را خودتان می‌نویسید.

نمایش خطای فیلد

Blazor (C#)	`<ValidationMessage For="() => model.Key" />`
Angular	`@if (form.controls.key.touched && form.controls.key.invalid)`
React	`{touched && errors.key && <small>...</small>}`
Vue	`<small v-if="touched && errors.key">`

Angular وضعیت touched را برای هر کنترل نگه می‌دارد؛ در بقیه یک پرچم ساده است.

ارسال

Blazor (C#)	`OnValidSubmit="Save"`
Angular	`(ngSubmit)="save()"`
React	`onSubmit={submit} + e.preventDefault()`
Vue	`@submit.prevent="submit"`

React و Vue باید جلوی ارسال پیش‌فرض مرورگر را بگیرند (preventDefault / .prevent).

کد

فرم ساخت پروژه

Blazor (C#) کد اجراشده و تست‌شده در /examples

examples/frontend/blazor/Lessons.Blazor/ProjectForm.razor

```
@using Microsoft.AspNetCore.Components.Forms
@inject ProjectsClient Api

@* EditForm + DataAnnotationsValidator run the [Required]/[RegularExpression] rules on the model. *@
<EditForm Model="model" OnValidSubmit="Save">
    <DataAnnotationsValidator />
    <label>Key <InputText @bind-Value="model.Key" /></label>
    <ValidationMessage For="() => model.Key" />
    <label>Name <InputText @bind-Value="model.DisplayName" /></label>
    <ValidationMessage For="() => model.DisplayName" />
    <label>Adapter URL <InputText @bind-Value="model.AdapterBaseUrl" /></label>
    <ValidationMessage For="() => model.AdapterBaseUrl" />
    @if (serverError is not null)
    {
        <p role="alert">@serverError</p>
    }
    <button type="submit" disabled="@saving">Save</button>
</EditForm>

@code {
    private CreateProjectModel model = new();
    private bool saving;
    private string? serverError;

    // Called only when every rule passes.
    private async Task Save()
    {
        saving = true;
        try
        {
            await Api.CreateAsync(model);
            model = new();
        }
        catch (Exception ex)
        {
            serverError = ex.Message; // the server checks again (e.g. 409 already exists)
        }
        finally
        {
            saving = false;
        }
    }
}
```

examples/frontend/web/angular/project-form.component.ts کد اجراشده و تست‌شده در /examples **Angular**

```
@Component({
  selector: 'app-project-form',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="save()">
      <label>Key <input formControlName="key" /></label>
      @if (form.controls.key.touched && form.controls.key.invalid) { <small>Lowercase letters, digits and
dashes, up to 40.</small> }
      <label>Name <input formControlName="displayName" /></label>
      @if (form.controls.displayName.touched && form.controls.displayName.invalid) { <small>Required, up to
100.</small> }
      <label>Adapter URL <input formControlName="adapterBaseUrl" /></label>
      @if (form.controls.adapterBaseUrl.touched && form.controls.adapterBaseUrl.invalid) { <small>An http(s)
URL.</small> }
      @if (serverError()) { <p role="alert">{{ serverError() }}</p> }
      <button [disabled]="form.invalid || saving()">Save</button>
    </form>
  `,
})
export class ProjectFormComponent {
  private api = inject(ProjectsService)
  saving = signal(false)
  serverError = signal<string | null>(null)

  // The rules live on the controls; the form is invalid while any control is.
  form = inject(FormBuilder).nonNullable.group({
    key: ['', [Validators.required, Validators.pattern(/^[a-z0-9-]{1,40}$/)]],
    displayName: ['', [Validators.required, Validators.maxLength(100)]],
    adapterBaseUrl: ['', [Validators.required, Validators.pattern(/^https?:\/\/\S+$/)]],
  })

  save() {
    if (this.form.invalid) return this.form.markAllAsTouched()
    this.saving.set(true)
    this.api
      .create(this.form.getRawValue())
      .pipe(finalize(() => this.saving.set(false))) // runs on success and on error
      .subscribe({
        next: () => this.form.reset(),
        // The server checks again; show its message (e.g. 409 already exists).
        error: (e) => this.serverError.set(e.error?.error ?? 'Could not save.'),
      })
  }
}
```

React کد اجراشده و تست‌شده در /examples examples/frontend/web/react/ProjectForm.tsx

```
export function ProjectForm() {
  const { create } = useProjects()
  // Controlled inputs: the value lives in state, every keystroke goes through setForm.
  const [form, setForm] = useState<CreateProject>({ key: '', displayName: '', adapterBaseUrl: '' })
  const [touched, setTouched] = useState(false)
  const [serverError, setServerError] = useState<string | null>(null)
  const [saving, setSaving] = useState(false)

  const errors = validateProject(form) // derived on each render, not stored
  const set = (field: keyof CreateProject) => (e: { target: { value: string } }) => setForm({ ...form, [field]:
e.target.value })

  async function submit(e: FormEvent) {
    e.preventDefault()
    setTouched(true)
    if (Object.keys(errors).length) return
    setSaving(true)
    try {
      await create(form)
      setForm({ key: '', displayName: '', adapterBaseUrl: '' })
      setTouched(false)
    } catch (err) {
      setServerError((err as Error).message) // the server checks again (e.g. 409 already exists)
    } finally {
      setSaving(false)
    }
  }

  return (
    <form onSubmit={submit} noValidate>
      <label>Key <input value={form.key} onChange={set('key')} /></label>
      {touched && errors.key && <small>{errors.key}</small>}
      <label>Name <input value={form.displayName} onChange={set('displayName')} /></label>
      {touched && errors.displayName && <small>{errors.displayName}</small>}
      <label>Adapter URL <input value={form.adapterBaseUrl} onChange={set('adapterBaseUrl')} /></label>
      {touched && errors.adapterBaseUrl && <small>{errors.adapterBaseUrl}</small>}
      {serverError && <p role="alert">{serverError}</p>}
      <button disabled={saving}>Save</button>
    </form>
  )
}
```

Vue کد اجراشده و تست‌شده در /examples `examples/frontend/web/vue/ProjectForm.vue`

```vue
<script setup lang="ts">
import { computed, reactive, ref } from 'vue'
import { validateProject, type CreateProject } from '../shared/model'
import { useProjects } from './useProjects'

const { create } = useProjects()
// v-model binds each input to a field of this reactive object, both ways.
const form = reactive<CreateProject>({ key: '', displayName: '', adapterBaseUrl: '' })
const touched = ref(false)
const saving = ref(false)
const serverError = ref<string | null>(null)
const errors = computed(() => validateProject(form)) // recomputed when a field changes

async function submit() {
  touched.value = true
  if (Object.keys(errors.value).length) return
  saving.value = true
  try {
    await create({ ...form })
    Object.assign(form, { key: '', displayName: '', adapterBaseUrl: '' })
    touched.value = false
  } catch (e) {
    serverError.value = (e as Error).message // the server checks again (e.g. 409 already exists)
  } finally {
    saving.value = false
  }
}
</script>

<template>
  <form novalidate @submit.prevent="submit">
    <label>Key <input v-model.trim="form.key" /></label>
    <small v-if="touched && errors.key">{{ errors.key }}</small>
    <label>Name <input v-model="form.displayName" /></label>
    <small v-if="touched && errors.displayName">{{ errors.displayName }}</small>
    <label>Adapter URL <input v-model.trim="form.adapterBaseUrl" /></label>
    <small v-if="touched && errors.adapterBaseUrl">{{ errors.adapterBaseUrl }}</small>
    <p v-if="serverError" role="alert">{{ serverError }}</p>
    <button :disabled="saving">Save</button>
  </form>
</template>
```

قواعد مشترک React و Vue

همه‌ی زبان‌ها کد اجراشده و تست‌شده در /examples `examples/frontend/web/shared/model.ts`

```
// The backend's rules, repeated on the client for instant feedback. The server still checks:
// client-side validation is for the user, not for security.
export function validateProject(p: CreateProject): Partial<Record<keyof CreateProject, string>> {
  const errors: Partial<Record<keyof CreateProject, string>> = {}
  if (!/^[a-z0-9-]{1,40}$/.test(p.key)) errors.key = 'Lowercase letters, digits and dashes, up to 40.'
  if (!p.displayName.trim()) errors.displayName = 'Required.'
  else if (p.displayName.length > 100) errors.displayName = 'Up to 100 characters.'
  if (!/^https?:\/\/\S+$/.test(p.adapterBaseUrl)) errors.adapterBaseUrl = 'An http(s) URL.'
  return errors
}
```

اشتباه‌های رایج

همه **اعتماد به اعتبارسنجی کلاینت برای امنیت**

چرا: درخواست را می‌شود مستقیم با curl فرستاد.

درست: سرور همیشه دوباره چک می‌کند (درس اعتبارسنجی بک‌اند).

همه **نادیده گرفتن پیام خطای سرور**

چرا: کاربر نمی‌فهمد چرا ذخیره نشد (مثلاً key تکراری).

درست: پیام error پاسخ را در فرم نشان بده.

Angular **subscribe با complete برای خاموش کردن دکمه‌ی در حال ذخیره**

چرا: در خطا complete صدا زده نمی‌شود و دکمه برای همیشه غیرفعال می‌ماند.

درست: finalize(() => saving.set(false)).

React **نگه داشتن errors در state جدا**

چرا: با تغییر فرم همگام نمی‌ماند و یک setState اضافه در هر تایپ.

درست: از form در هر render محاسبه کن.

Blazor (C#) **OnSubmit به‌جای OnValidSubmit**

چرا: با OnSubmit اعتبارسنجی را خودت باید صدا بزنی؛ فرم نامعتبر هم ارسال می‌شود.

درست: OnValidSubmit، یا editContext.Validate() در OnSubmit.

این کد کجای پروژه‌هاست

`ControlPanel/services/core-api/src/projects/dto/project.dto.ts:4`

قواعد سرور که فرم تکرار می‌کند

`ClinicReport/web/src/pages/Login.tsx:13`

React: input کنترل‌شده با useState

تمرین

▼ 1. در Blazor، کلاس CreateProjectModel را به یک پروژه‌ی Shared ببر و هم در سرور و هم در کلاینت استفاده کن. چه چیزی به دست می‌آوری؟

پاسخ: یک تعریف قواعد برای هر دو طرف؛ اگر قاعده عوض شود، فرم و API با هم عوض می‌شوند. این در React و Vue فقط با کد TS مشترک (مثلاً schema zod در یک monorepo) شدنی است.

▼ 2. در React، هم‌زمان با تایپ key فقط حروف کوچک را بپذیر (حروف بزرگ را کوچک کن). کجا تغییر می‌دهی؟

پاسخ: در (() e.target.value.toLowerCase(), key: ...form }) :setForm('key'). چون input کنترل‌شده است، مقدار نمایش‌داده‌شده هم کوچک می‌شود.

▼ 3. دکمه‌ی Save را تا زمانی که فرم نامعتبر است غیرفعال کن، در Vue. خوب است یا بد؟

پاسخ: :disabled="saving || Object.keys(errors).length > 0". کار می‌کند، ولی کاربر نمی‌فهمد چرا دکمه غیرفعال است تا پیام‌ها را ببیند؛ نمایش پیام‌ها بعد از اولین کلیک معمولاً بهتر است.

جمله‌ی مصاحبه

The form repeats the server's rules for instant feedback, shows errors after the first attempt, and still displays the server's answer, because the server is the real check.